

Prepare a integração

Adicione um botão ao seu aplicativo. O BatePonto abre em tela cheia, conduz o login e a navegação e permite retornar ao hospedeiro. **Siga os passos comuns e depois a trilha Android ou Flutter.**

Confira os requisitos

Use um aplicativo Android ou Flutter que já compile. Esta é a configuração de referência dos exemplos:

Configuração	Valor
Plataforma / arquitetura	Android / arm64-v8a
Android SDK	compileSdk e targetSdk 36; minSdk 26
Java / alvo JVM / Kotlin	17 / 17 / 2.1.20
Android Gradle Plugin	8.12.0
Gradle	Nativo: 9.0.0 · Flutter: 8.13
Flutter / Dart	3.24.5 / 3.5.4 no exemplo Flutter

Limites desta prévia: execução exercitada em emulador API 36 ARM64 com páginas de 4 KB. Outras versões, aparelhos físicos e páginas de 16 KB precisam de homologação. O SDK declara mínimo API 24; os exemplos usam 26. Nenhum desses mínimos comprova toda a matriz Android.

Hospedeiro com **React Native** requer adaptação; integração direta não suportada. **Firebase próprio** exige análise de coexistência antes de integrar. Push do BatePonto está desativado. iOS, x86 e ARM de 32 bits não estão incluídos.

Organize os arquivos

Extraia bateponto-sdk-android.zip para obter a estrutura abaixo. Preserve a pasta **Maven inteira**, incluindo dependências, .pom e .module; apenas o AAR principal não basta.

Terminal

```
mkdir -p integracao/bateponto-sdk
unzip /caminho/para/bateponto-sdk-android.zip -d integracao/bateponto-sdk
```

Estrutura

```
integracao/
├── bateponto-sdk/maven/
├── meu-app-android/
│   ├── settings.gradle.kts
│   └── app/build.gradle.kts
└── meu-app-flutter/
    └── android/
        ├── settings.gradle.kts
        └── app/build.gradle.kts
```

O primeiro build pode precisar da internet para dependências públicas. O consumidor não precisa dos fontes BatePonto, Node ou Metro.

Antes de prosseguir: confira [compatibilidade](#) e, se seu app usa Firebase, [permissões e manifesto](#).

Configure o módulo Android

Passos comuns às duas trilhas. Os trechos usam **Kotlin DSL**: incorpore-os aos blocos existentes, preservando plugins, namespace, applicationId e assinatura.

1. Registre o Maven e a dependência

Em `settings.gradle.kts` — dentro de `android/` no Flutter — adicione o Maven antes dos repositórios públicos:

```
Kotlin

dependencyResolutionManagement {
    repositories {
        maven { url = uri("../bateponto-sdk/maven") }
        google()
        mavenCentral()
    }
}
```

No Flutter, troque o caminho por `../bateponto-sdk/maven`. Para Groovy (`.gradle`) ou repositórios em `allprojects`, consulte a trilha [Android nativo](#) ou [Flutter](#). Preserve `pluginManagement` e o carregador Flutter; não duplique políticas de repositório.

Em `app/build.gradle.kts` — `android/app/build.gradle.kts` no Flutter — adicione ao bloco `dependencies`:

```
Kotlin

implementation("com.pontotel.bateponto:sdk:0.1.0")
```

2. Ajuste Android, Java e Kotlin

No mesmo arquivo do módulo, usando as versões da página anterior:

```
Kotlin

android {
    compileSdk = 36
    defaultConfig {
        minSdk = 26
        targetSdk = 36
        ndk { abiFilters += "arm64-v8a" }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
}

kotlin {
    compilerOptions {
        jvmTarget.set(org.jetbrains.kotlin.gradle.dsl.JvmTarget.JVM_17)
    }
}
```

Em projeto somente Java, omita o bloco `kotlin`. Garanta **somente ARM64** nos filtros e splits da variante; o trecho não remove outras ABIs já configuradas.

Em `gradle.properties` na raiz Gradle, habilite `android.useAndroidX=true`. No Flutter, o arquivo fica em `android/`; o exemplo validado também usa `android.enableJetifier=true`.

Abra pelo Android ou crie a ponte Flutter

Android nativo · conecte seu botão

Na Activity visível do aplicativo, com um botão já existente:

Kotlin

```
import com.pontotel.bateponto.sdk.BatePontoSdk

// Dentro da Activity do hospedeiro.
abrirBatePontoButton.setOnClickListener {
    BatePontoSdk.open(this)
}
```

Use uma **Activity válida na thread principal**. O manifesto do SDK já fornece sua Activity; não é necessário declará-la novamente nem alterar sua Application. Veja a [chamada Java](#) se necessário. **Android nativo: prossiga para “Compile e valide”**.

Flutter · registre o canal Kotlin

Em android/app/src/main/kotlin/.../MainActivity.kt, adapte o pacote e incorpore o canal à Activity existente. Preserve seus outros canais, plugins e comportamento. Exemplo mínimo:

Kotlin

```
package com.exemplo.meuapp

import com.pontotel.bateponto.sdk.BatePontoSdk
import io.flutter.embedding.android.FlutterActivity
import io.flutter.embedding.engine.FlutterEngine
import io.flutter.plugin.common.MethodChannel

class MainActivity : FlutterActivity() {
    override fun configureFlutterEngine(flutterEngine: FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)
        MethodChannel(
            flutterEngine.dartExecutor.binaryMessenger,
            "com.pontotel.bateponto/sdk"
        ).setMethodCallHandler { call, result ->
            if (call.method != "open") {
                result.notImplemented()
            } else {
                try {
                    BatePontoSdk.open(this)
                    result.success(null)
                } catch (error: Exception) {
                    result.error("OPEN_FAILED",
                        "Não foi possível abrir o BatePonto.", null)
                }
            }
        }
    }
}
```

OPEN_FAILED trata uma exceção síncrona da solicitação de abertura; não captura falhas posteriores dentro do SDK. Continue no Dart na próxima página.

Conecte o botão Flutter

Encaminhe a ação pelo MethodChannel

Exemplo de tela mínima em Dart. Use este botão em uma tela com Scaffold; em aplicativos existentes, preserve os controles e o tratamento de estado da sua interface.

Dart

```
import 'package:flutter/foundation.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

class BatePontoEntry extends StatelessWidget {
  const BatePontoEntry({super.key});
  static const _channel = MethodChannel('com.pontotel.bateponto/sdk');

  Future<void> _open(BuildContext context) async {
    String? message;
    try {
      await _channel.invokeMethod<void>('open');
    } on PlatformException {
      message = 'Não foi possível abrir o BatePonto.';
    } on MissingPluginException {
      message = 'A ponte Android do BatePonto não está disponível.';
    }
    if (message != null && context.mounted) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text(message)),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    final supported = !kIsWeb &&
      defaultTargetPlatform == TargetPlatform.android;
    return FilledButton(
      onPressed: supported ? () => _open(context) : null,
      child: const Text('Abrir BatePonto'),
    );
  }
}
```

Insira `const BatePontoEntry()` no corpo de uma tela. O canal `com.pontotel.bateponto/sdk` e o método `open` devem coincidir exatamente com a ponte Kotlin. Não há pacote `BatePonto` a adicionar ao `pubspec.yaml`.

Para um botão com estado “Abrindo...” e bloqueio enquanto a solicitação está pendente, use o [componente completo](#), também explicado na [referência Flutter](#).

Entenda o retorno da chamada

O `await` termina quando a ponte responde à **solicitação de abertura**. Ele não espera o usuário sair do `BatePonto` e não confirma login ou marcação. Esta versão não oferece callbacks de negócio, SSO nem API pública de logout.

Depois de alterar Kotlin ou dependências Android, faça um **build completo e reinstale o aplicativo**. Hot reload não incorpora essas mudanças.

Compile e valide

Gere o aplicativo hospedeiro

Android nativo, a partir de meu-app-android/:

Terminal

```
./gradlew :app:assembleDebug
```

Flutter, a partir de meu-app-flutter/:

Terminal

```
flutter pub get  
flutter build apk --debug --target-platform android-arm64
```

Instale o APK em um dispositivo ou emulador **ARM64**. O SDK usa o bundle embarcado; não precisa de Metro. O backend continua sendo o ambiente configurado na distribuição: build local não significa servidor local. Confirme o ambiente recebido antes de inserir dados.

Checklist da primeira integração

1. Toque no botão e confira a abertura do BatePonto em tela cheia.
2. Verifique as permissões necessárias e o comportamento ao negar acesso.
3. Abra e cancele a câmera; confirme que o SDK continua funcionando.
4. Use Voltar até a raiz e saia; confira o retorno ao hospedeiro.
5. Reabra o SDK; confira a sessão e a navegação. **Fechar não faz logout.**
6. Valide também o build release com assinatura, minificação e integrações reais do seu app.

O login acontece dentro do BatePonto. Homologue login, marcação e reconhecimento facial em aparelhos reais antes de distribuir. O motor Luxand está incluído; a validação desta entrega não comprova biometria completa nem marcação real.

Nesta geração, Bifrost fornece a configuração de Luxand e logs quando as chaves e o token do usuário estão disponíveis. Até o cadastro das chaves, o módulo usa a contingência local, também usada na primeira abertura offline. O app Flutter não precisa configurar Bifrost nem fornecer chaves. A geração com Bifrost passou nos builds e auditorias, mas ainda não foi executada em dispositivo.

Se algo impedir o primeiro teste

Sintoma	Primeiro passo
Dependência não encontrada	Confira caminho, Maven inteiro e repositórios públicos.
MissingPluginException	Confira canal/método e recompile/reinstale o app Flutter.
Erro nativo ou ABI incompatível	Use ARM64 e confira filtros e splits do hospedeiro.

Consulte a referência completa

Abra o [guia do integrador](#) ou o [PDF](#) para API, manifesto, diagnóstico e segurança. O SDK compartilha processo e pacote com o hospedeiro.